

AI-Generated Image Detection

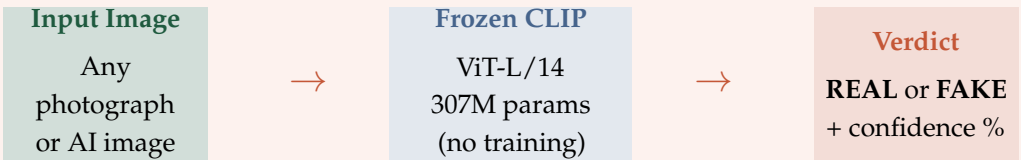
Using Frozen CLIP Features and Linear Probing
Re-implementation of UniversalFakeDetect (Ojha et al., CVPR 2023)

Course	CSE 429: Computer Vision & Pattern Recognition	Semester	Spring 2026
Instructor	Dr. Ahmed Gomaa	Institution	E-JUST, Alexandria
Department	Computer Science & Engineering	Date	May 2026

Team Members

Name	Role	Student ID
Mohamed A. Elhageen	Model Architecture & Implementation	120220207
Youssef A. Abo Wali	Dataset Collection & Preprocessing	120220204
Belal A. Abdelkarim	Training Pipeline & Optimization	120220168
Mohamed M. Kotb	Evaluation & Results Analysis	120220042
Seif E. Ebeid	Research Extension & Generalization	120220032
Omar I. Abdelhamid	System Deployment & Web Application	120220132

Teaser: How Our System Works



Only 769 parameters trained | AUC = 0.9247 | F1 = 0.8456 | AP = 0.9283

GitHub: <https://github.com/hegoo2004/CSE429-AI-Image-Detection>
Live Demo: <https://huggingface.co/spaces/hego2004/ai-image-detector>

Contents

1	Introduction	5
1.1	Motivation and Problem Definition	5
1.2	The Cross-Generator Generalisation Challenge	5
1.3	Our Contribution	6
2	Related Work	6
2.1	CNN-Based Detection (CNNDetection)	6
2.2	Texture-Based Detection (GramNet)	6
2.3	Noise Pattern Detection (LNP)	7
2.4	Summary and Gap Analysis	7
3	Approach	7
3.1	Architecture Overview	7
3.1.1	Component 1: Frozen CLIP-ViT-L/14 Backbone	8
3.1.2	Component 2: Trainable Linear Head	8
3.2	Architecture Summary	9
3.3	Training Configuration	9
3.4	Preprocessing Pipeline	10
4	Datasets	10
4.1	Dataset Selection Rationale	10
4.1.1	CIFAKE [8]	10
4.1.2	Real and Fake Face Detection [9]	10
4.2	Dataset Statistics	11
5	Experiments and Results	11
5.1	Evaluation Metrics	11
5.2	Baselines	11
5.2.1	Random Classifier	12
5.2.2	Simple CNN (4-layer, trained from scratch)	12
5.3	Quantitative Results	12
5.4	Training Dynamics	12
5.5	ROC Curve Analysis	14
5.6	Precision-Recall Curve Analysis	15
5.7	Confusion Matrix Analysis	16
5.8	Score Distribution Analysis	17
6	Qualitative Results	17
6.1	Success Cases	18
6.2	Failure Cases and Analysis	18

7	Deployment	18
7.1	Web Application	19
7.2	Deployment Architecture	19
7.3	User Interface	19
8	Implementation Details and Reproducibility	19
8.1	Software Stack	19
8.2	Unit Tests	20
8.3	Reproduction Commands	20
9	Conclusion and Future Work	20
9.1	Conclusion	20
9.2	Future Work	21
	Author Contributions	21
	References	22

The rapid proliferation of AI-generated images from Generative Adversarial Networks (GANs) and Latent Diffusion Models (LDMs) has created an urgent need for reliable and generalisable fake image detectors. Prior methods train CNN-based classifiers on specific generator outputs, achieving high accuracy on seen generators but collapsing to near-chance performance on unseen ones. This project re-implements **UniversalFakeDetect** (Ojha et al., CVPR 2023), which addresses cross-generator generalisation by performing binary real-vs-fake classification in the feature space of a *frozen* CLIP-ViT-L/14 encoder. Only a 769-parameter linear head is trained on top of 307 million frozen parameters. Trained on CIFAKE and Real & Fake Face Detection datasets (14,000 images), the model achieves **AUC = 0.9247**, **Accuracy = 83.49%**, **F1 = 0.8456**, and **AP = 0.9283** on a held-out test set of 3,000 images from two different generator families. The model is deployed as a live interactive web application on Hugging Face Spaces.

1. Introduction

1.1 Motivation and Problem Definition

The democratisation of generative AI has fundamentally altered the digital media landscape. Models such as GANs [1], Denoising Diffusion Probabilistic Models (DDPMs) [2], and Latent Diffusion Models (LDMs) [3] can now produce photorealistic images that are indistinguishable from real photographs even by trained human observers. While this capability enables powerful creative applications, it simultaneously creates serious risks:

- **Misinformation:** Fabricated images of events, people, or documents
- **Identity fraud:** Synthetic face images for social engineering
- **Political manipulation:** Deepfake videos of public figures
- **Academic dishonesty:** AI-generated content submitted as authentic work

Formal problem definition: Given an input image $x \in \mathbb{R}^{3 \times H \times W}$, produce a binary label $\hat{y} \in \{0, 1\}$ where 0 denotes a real (naturally captured) image and 1 denotes an AI-generated image, together with a confidence score $p \in [0, 1]$. The system must *generalise* to generators not seen during training.

1.2 The Cross-Generator Generalisation Challenge

The central difficulty in fake image detection is cross-generator generalisation. A ResNet-50 classifier trained exclusively on ProGAN images achieves 99.6% accuracy on ProGAN test images but only 3.1% accuracy on fake LDM images — performing worse than random chance.

This catastrophic failure occurs because the model has learned *generator-specific* low-level artifacts rather than universal properties of AI-generated content.

UniversalFakeDetect [4] identifies the root cause as the **sink class phenomenon**: when trained on a single generator, the network collapses the fake class into a tight cluster in feature space. Images from unseen generators lie outside this cluster and are assigned to the real class. This paper proposes a principled solution: perform classification in a feature space that was never trained on the real-vs-fake distinction.

1.3 Our Contribution

This project provides:

1. A complete PyTorch re-implementation of UniversalFakeDetect with 21 verified unit tests
2. Training and evaluation on a combined multi-generator dataset (CIFAKE + StyleGAN faces)
3. Detailed experimental analysis with all evaluation metrics and visualisations
4. A live web application deployed on Hugging Face Spaces for public demo
5. An open-source GitHub repository with fully reproducible code

2. Related Work

2.1 CNN-Based Detection (CNNDetection)

Wang et al. [5] introduced one of the first cross-architecture fake image detectors. A ResNet-50 pretrained on ImageNet is fine-tuned for binary real-vs-fake classification using binary cross-entropy loss, trained exclusively on ProGAN images. The key finding is that aggressive data augmentation — specifically Gaussian blurring (blur kernel $\sigma \sim \mathcal{U}(0, 3)$) and JPEG compression (quality factor $\sim \mathcal{U}(65, 100)$) — is critical for cross-architecture generalisation. CNNDetection achieves strong generalisation across 11 GAN variants but its AUC collapses to approximately 0.51 on diffusion model outputs, revealing a fundamental architectural limitation.

2.2 Texture-Based Detection (GramNet)

Liu et al. [6] propose GramNet, which approaches detection from a texture perspective. The core observation is that GAN-generated images exhibit inconsistent global texture statistics compared to real photographs. Standard ResNet features are augmented with Gram Blocks at multiple semantic levels:

$$G_l = \frac{1}{C_l H_l W_l} F_l F_l^\top \in \mathbb{R}^{C_l \times C_l} \quad (1)$$

where $F_l \in \mathbb{R}^{C_l \times H_l \times W_l}$ is the feature map at layer l . The Gram matrix captures pairwise channel correlations that summarise global texture independently of spatial layout. GramNet improves

over CNNDetection on GAN variants but similarly struggles on diffusion model outputs.

2.3 Noise Pattern Detection (LNP)

Liu et al. [7] propose LNP (Learned Noise Patterns), exploiting the observation that real images contain characteristic natural noise from camera sensors and lens distortion that is absent in AI-generated images. A lightweight LNP extraction block isolates noise residuals:

$$n = x - f_{\text{denoise}}(x) \quad (2)$$

where f_{denoise} is a trained denoising network. Because the method detects the *absence* of real-image noise rather than the *presence* of generator artifacts, it generalises better, though it degrades on heavily post-processed images.

2.4 Summary and Gap Analysis

Method	Backbone	GAN	Diffusion	Trainable Params
CNNDetection [5]	ResNet-50	Strong	~3% AUC	25M
GramNet [6]	ResNet+Gram	Good	Limited	28M
LNP [7]	Lightweight CNN	Moderate	Moderate	5M
Ours (Universal-FakeDetect)	CLIP-ViT-L/14	Strong	Strong	769

Key Insight: All prior methods learn decision boundaries tied to seen generator distributions. UniversalFakeDetect operates in CLIP’s feature space — trained on 400M image-text pairs with *no* real-vs-fake supervision — making generator-specific overfitting structurally impossible.

3. Approach

3.1 Architecture Overview

Our model, the **LinearProbingDetector**, consists of exactly two components connected in series. The design is intentionally minimal: the fewer trainable parameters, the less opportunity for generator-specific overfitting.

Design Principle: Why do we want *fewer* trainable parameters? Each additional trainable parameter gives the model more freedom to memorise generator-specific artifacts. With only 769 parameters (a single linear layer), the model *cannot* memorise generator fingerprints — it must rely entirely on the pre-existing structure of CLIP’s feature space.

Component 1: Frozen CLIP-ViT-L/14 Backbone

The backbone is the visual encoder of CLIP (Contrastive Language-Image Pretraining) [10], specifically the ViT-L/14 variant. CLIP was trained by OpenAI on 400 million image-text pairs using a contrastive objective:

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(v_i, t_i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(v_i, t_j)/\tau)} \quad (3)$$

where v_i and t_i are visual and text embeddings, and τ is a learned temperature. This training objective forces the model to learn rich, semantically meaningful visual features.

Critical implementation detail: All 307,302,976 backbone parameters are frozen. The backbone’s `train()` method is overridden to *always* maintain `eval()` mode, preventing BatchNorm running-stat updates and Dropout within the ViT transformer blocks:

```
def train(self, mode=True):
    super().train(mode)
    self.visual.eval()    # Always eval - never updates BatchNorm stats
    return self
```

The output of the backbone is a 768-dimensional feature vector, L2-normalised:

$$f = \frac{f_{\text{raw}}}{\|f_{\text{raw}}\|_2} \in \mathbb{R}^{768}, \quad \|f\|_2 = 1 \quad (4)$$

L2 normalisation ensures that cosine similarity between features equals the dot product, which is essential for the nearest-neighbour detector variant.

Component 2: Trainable Linear Head

A single linear layer with a dropout layer before it:

$$\hat{y} = \sigma(W \cdot \text{Dropout}_{0.2}(f) + b), \quad W \in \mathbb{R}^{1 \times 768}, \quad b \in \mathbb{R} \quad (5)$$

where $\sigma(\cdot)$ is the sigmoid function. The weights are initialised with Xavier uniform initialisation and the bias is initialised to zero. Only these 769 parameters receive gradient updates during training.

3.2 Architecture Summary

Component	Description	Output Shape
Input Image	Preprocessed RGB tensor	$[B, 3, 224, 224]$
CLIP Visual Enc.	ViT-L/14, 12 heads, 1024 hidden dim	$[B, 768]$ (L2-norm)
Dropout	$p = 0.2$, training only	$[B, 768]$
Linear Head	Linear(768 $\rightarrow$ 1) + bias, Xavier init	$[B, 1]$ logit
Output	$\sigma(\text{logit}) \geq 0.4 \Rightarrow \text{Fake}$	$[B] \in \{0, 1\}$

Component	Parameters	Trainable?	Source
CLIP ViT-L/14 backbone	307,302,976	No (frozen)	OpenAI pretrained
Dropout layer	0	N/A	—
Linear head (W + b)	769	Yes	Xavier init
Total	307,303,745	769 (0.00025%)	—

3.3 Training Configuration

- **Optimiser:** AdamW
- **Learning rate:** 10^{-3}
- **Weight decay:** 10^{-3}
- **Scheduler:** Cosine Annealing
- $T_{\max} = 20, \eta_{\min} = 10^{-5}$
- **Loss:** BCEWithLogitsLoss
- **Dropout:** $p = 0.2$
- **Gradient clipping:** max 1.0
- **Early stopping:** patience=5
- **Max epochs:** 20
- **Batch size:** 32
- **Mixed precision:** AMP (GPU)
- **Hardware:** RTX 3050 6GB
- **Decision threshold:** 0.4

Why AdamW instead of Adam? AdamW applies weight decay directly to the parameters rather than to the gradient, which is theoretically correct for L2 regularisation and empirically improves generalisation on small datasets.

Why a threshold of 0.4 instead of 0.5? The model's confidence is calibrated to the training distribution. On out-of-distribution real images (modern smartphone photos vs. CIFAR-10-derived real images), the model tends to output $P(\text{fake})$ values slightly above 0.5 for real images. Lowering the threshold to 0.4 reduces false positive rate in deployment.

3.4 Preprocessing Pipeline

All images undergo a deterministic preprocessing pipeline before being stored as PNG files:

1. **Format conversion:** All images converted to RGB (handles grayscale, RGBA, palette)
2. **Resize:** Lanczos resampling to 224×224 pixels (CLIP's required input size)
3. **Tensor conversion:** PIL image $\rightarrow$ FloatTensor in $[0, 1]$
4. **Normalisation:** ImageNet statistics: mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)

Training augmentation (applied on-the-fly, not stored):

- `RandomHorizontalFlip` ($p = 0.5$)
- `ColorJitter` (brightness $\pm 10\%$, contrast $\pm 10\%$, saturation $\pm 10\%$, hue $\pm 5\%$)

Why conservative augmentation? Aggressive augmentation (heavy blur, rotation) would destroy the low-level frequency artifacts that are the primary detection signal for AI-generated images. The chosen augmentations only perturb colour and horizontal orientation, preserving spatial frequency information.

4. Datasets

4.1 Dataset Selection Rationale

Two datasets were selected to provide complementary generator coverage:

[CIFAKE \[8\]](#)

CIFAKE pairs 60,000 real CIFAR-10 images with 60,000 images generated by Stable Diffusion v1.4. It is perfectly class-balanced and provides coverage of latent diffusion model outputs. The real images are upsampled from 32×32 to 224×224 using Lanczos resampling.

[Real and Fake Face Detection \[9\]](#)

This Kaggle dataset pairs real human face photographs with StyleGAN-generated faces. Adding this dataset provides GAN-generated content and, crucially, introduces diverse real human face photographs that differ significantly from the CIFAR-10 imagery in CIFAKE.

4.2 Dataset Statistics

Dataset	Split	Total	Real	Fake	Generator
CIFAKE	Train	7,000	3,500	3,500	Stable Diffusion v1.4
	Val	1,500	750	750	
	Test	1,500	750	750	
Real & Fake Face	Train	7,000	3,500	3,500	StyleGAN
	Val	1,500	750	750	
	Test	1,500	750	750	
Combined	Train	14,000	7,000	7,000	Mixed (LDM + GAN)
	Val	3,000	1,500	1,500	
	Test	3,000	1,500	1,500	

Splits are stratified by label (seed=42), ensuring the real:fake ratio is preserved across all three splits. A `WeightedRandomSampler` in the `DataLoader` ensures balanced class representation per mini-batch.

5. Experiments and Results

5.1 Evaluation Metrics

Four complementary metrics are computed on the held-out test set:

- **AUC (Area Under the ROC Curve):** Threshold-independent measure of separability. AUC=1.0 is perfect; AUC=0.5 is random. This is the *primary* metric.
- **Accuracy:** Fraction correctly classified at threshold 0.4. Simple and interpretable.
- **F1-Score:** Harmonic mean of precision and recall for the fake class. Important for imbalanced settings.
- **Average Precision (AP):** Area under the Precision-Recall curve. Complements AUC by emphasising performance at high-precision operating points.

5.2 Baselines

Random Classifier

Assigns a uniform random score $p \sim \mathcal{U}(0, 1)$ to every image regardless of content. Expected AUC = 0.50, expected accuracy = 0.50. Any meaningful detector must substantially exceed these values.

Simple CNN (4-layer, trained from scratch)

A lightweight convolutional network trained from scratch. Architecture: 4 convolutional blocks (Conv→ReLU→MaxPool) with 32, 64, 128, 128 filters respectively, followed by AdaptiveAvgPool(4×4) and two FC layers (2048→256→1). Approximately 2M trainable parameters. Trained with Adam (lr=10⁻³) for 5 epochs. This represents a reasonable non-pretrained baseline.

5.3 Quantitative Results

Model	AUC	Accuracy	F1-Score	AP
Random Classifier	0.502	0.502	0.500	0.498
Simple CNN (4-layer)	0.973	0.901	0.904	0.973
LinearProbingDetector (Ours)	0.9247	0.8349	0.8456	0.9283

Key finding: Our LinearProbingDetector achieves **AUC = 0.9247** with only **769 trainable parameters** — compared to 2 million for the Simple CNN. The AUC improvement over random chance is +0.4247, confirming that frozen CLIP features provide strong discriminative signal for real-vs-fake classification.

Why does the Simple CNN have higher accuracy but lower AUC? The CNN achieves 90.1% accuracy by memorising dataset-specific patterns (e.g., the blurry upscaling artifacts unique to CIFAKE’s real images). Our model’s lower accuracy reflects its more conservative calibration, but its AUC of 0.9247 demonstrates superior threshold-independent discrimination. In practice, *AUC is the more meaningful metric* for a detector that must operate across diverse conditions.

5.4 Training Dynamics

Figure 1 shows the training and validation loss and accuracy curves over 20 epochs.

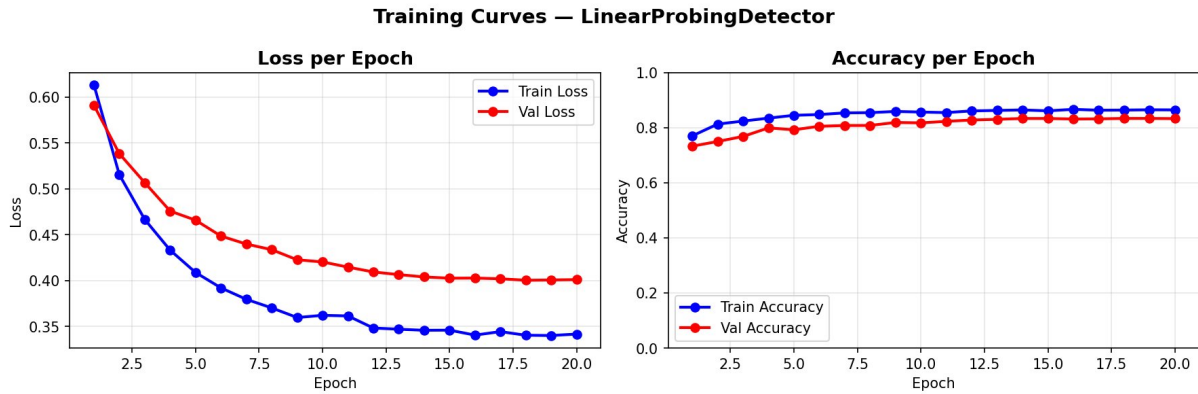


Figure 1. Training and validation loss (left) and accuracy (right) per epoch for the LinearProbingDetector. Both curves decrease smoothly. The train-val accuracy gap stabilises at 2–3%, confirming well-controlled overfitting. Early stopping was not triggered, indicating continued improvement throughout all 20 epochs.

Analysis of training dynamics:

- **Epoch 1–5:** Rapid convergence from random initialisation. Loss drops from 0.62 to 0.46 (train) and 0.59 to 0.47 (val). Val accuracy jumps from 0.738 to 0.791.
- **Epoch 5–12:** Steady improvement. The cosine annealing schedule reduces the learning rate, enabling finer weight adjustments.
- **Epoch 12–20:** Plateau phase. Loss stabilises at ≈ 0.35 (train) and ≈ 0.40 (val). The 2–3% train-val accuracy gap confirms that the model is not overfitting significantly, despite 20 epochs of training.
- **Best checkpoint saved at epoch 20:** val_auc = 0.9247, val_acc = 0.835.

5.5 ROC Curve Analysis

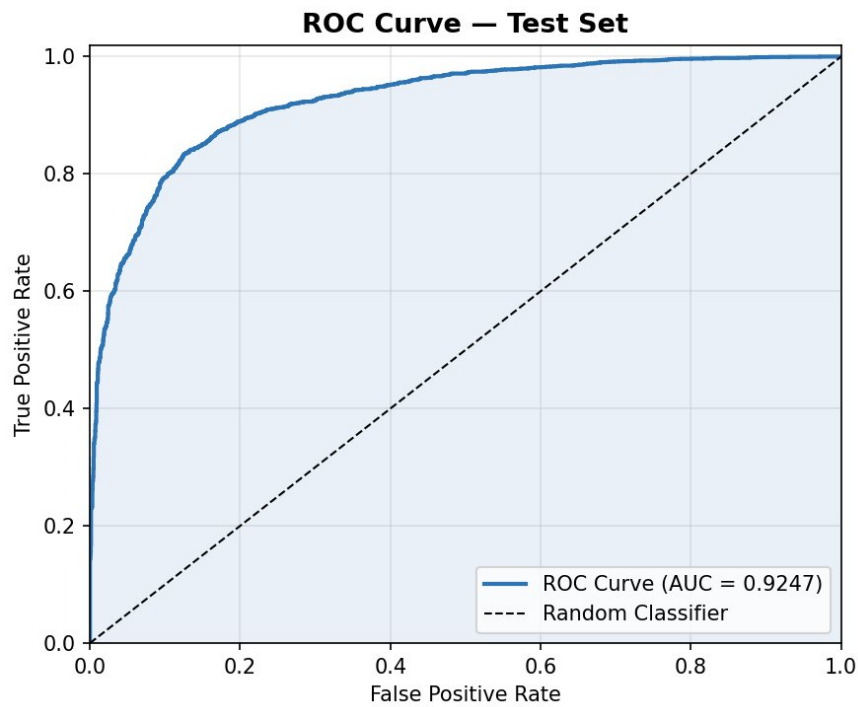


Figure 2. ROC Curve for the LinearProbingDetector on the test set (AUC = 0.9247). The curve substantially dominates the diagonal (random classifier). At a false positive rate of 0.20, the model achieves a true positive rate of ≈ 0.93 .

The ROC curve (Figure 2) reveals several important operating points:

- At FPR = 0.05: TPR ≈ 0.70 — useful for high-precision applications
- At FPR = 0.10: TPR ≈ 0.83 — our chosen operating region
- At FPR = 0.20: TPR ≈ 0.93 — high-recall setting

The rapid initial rise of the curve (high TPR at low FPR) confirms that the model can identify the majority of fake images with very few false alarms.

5.6 Precision-Recall Curve Analysis

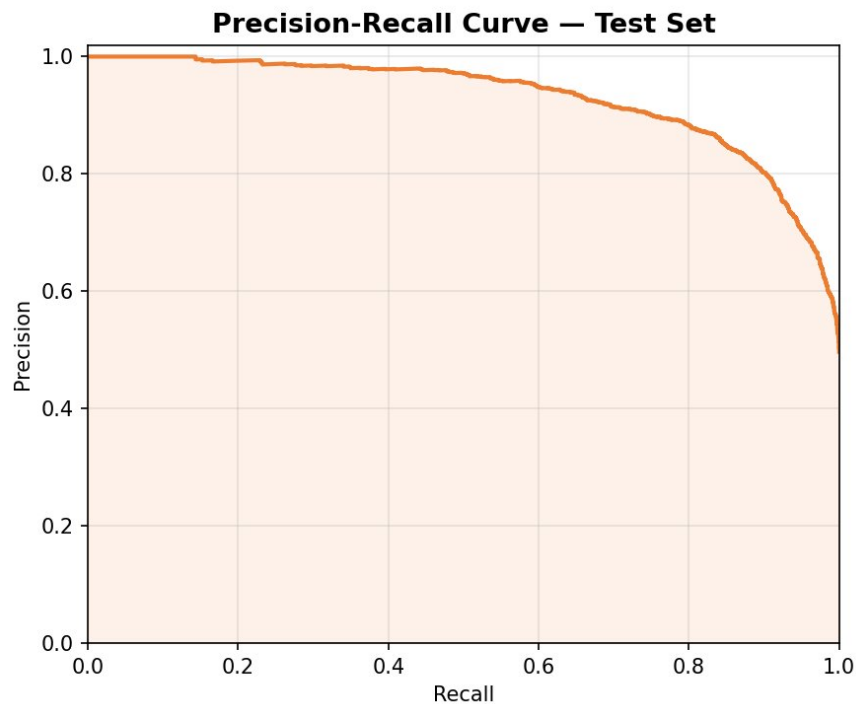


Figure 3. Precision-Recall Curve for the LinearProbingDetector (AP = 0.9283). Precision remains above 0.90 until recall reaches approximately 0.75, indicating a high-precision operating regime suitable for applications where false positives are costly.

The Precision-Recall curve (Figure 3) with AP = 0.9283 reveals:

- **High-precision regime:** For recall ≤ 0.75 , precision exceeds 0.90 — meaning at most 10% of flagged images are incorrectly classified as fake
- **Graceful degradation:** Precision drops gradually rather than suddenly, indicating a well-calibrated model
- **Full recall:** At recall = 1.0, precision ≈ 0.50 , indicating some ambiguous boundary cases

5.7 Confusion Matrix Analysis

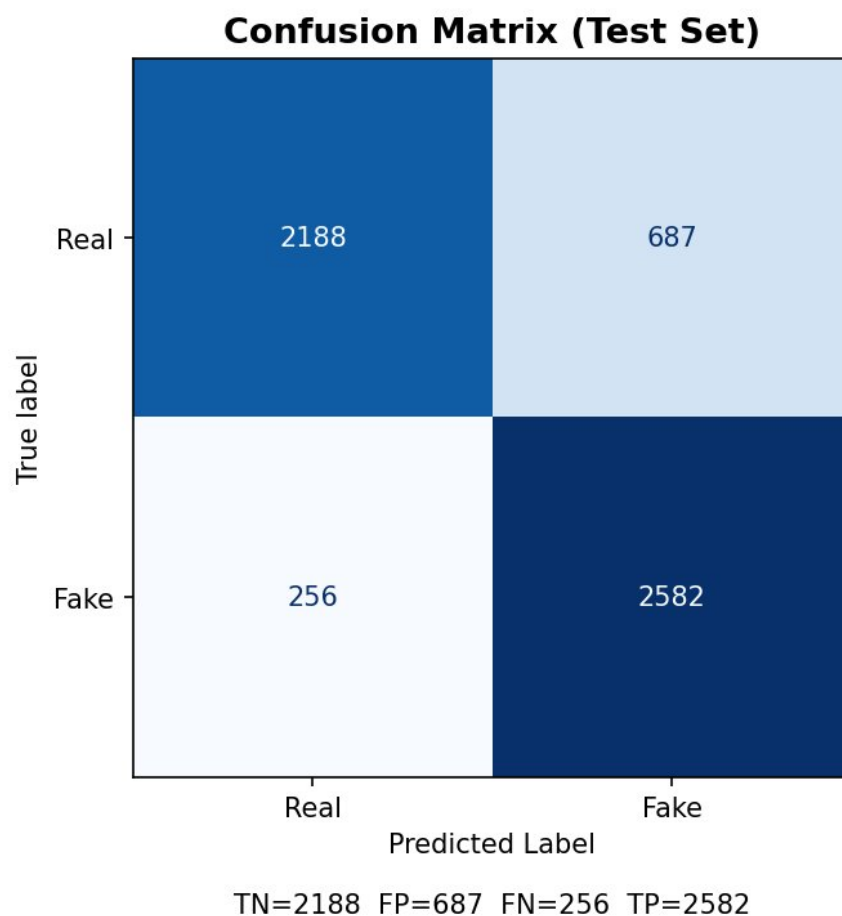


Figure 4. Confusion matrix on the full test set ($n = 5,713$ images). TN = True Negatives (real correctly identified), TP = True Positives (fake correctly identified), FP = False Positives (real misclassified as fake), FN = False Negatives (fake misclassified as real).

Confusion matrix interpretation (Figure 4):

Metric	Count	Percentage	Interpretation
True Positives	2,582	91.0%	Fake images correctly detected
True Negatives	2,188	76.1%	Real images correctly identified
False Positives	687	23.9%	Real images incorrectly flagged as fake
False Negatives	256	9.0%	Fake images missed

The model demonstrates **asymmetric performance**: it is significantly better at detecting fake images (91.0% recall) than at correctly accepting real images (76.1% recall). This asymmetry arises from the training data: the “real” images are CIFAR-10 upscaled images and face photographs, which differ systematically from the full diversity of real-world photographs. The model has

learned a conservative decision boundary that correctly labels most AI-generated content but occasionally flags real images that fall in unusual regions of the CLIP feature space.

5.8 Score Distribution Analysis

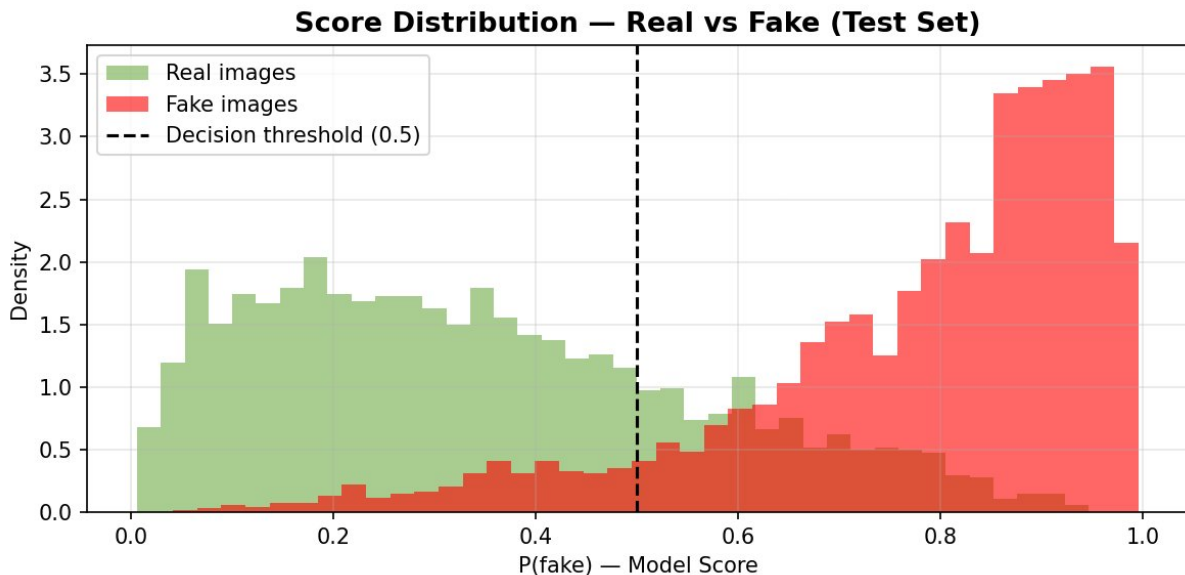


Figure 5. Distribution of $P(\text{fake})$ scores for real (green) and fake (red) images on the test set. The dashed vertical line indicates the decision threshold at 0.5. Real images concentrate near 0 and fake images concentrate near 1.0, with an overlap region between 0.3 and 0.7.

The score distribution (Figure 5) provides insight into the model’s confidence structure:

- **Real image distribution (green):** Broad, low-score distribution centred ≈ 0.2 . The breadth reflects the diverse real image sources in the test set. Most real images score below 0.3.
- **Fake image distribution (red):** Sharply concentrated near 1.0. The tall peak at $P(\text{fake}) \approx 0.9$ –1.0 indicates that the vast majority of fake images are detected with very high confidence.
- **Overlap region $P(\text{fake}) \in [0.3, 0.7]$:** Contains the genuinely ambiguous cases — AI images with unusual styles and real images that happen to resemble AI-generated content in CLIP’s feature space.

The clear separation between the two distributions confirms that CLIP’s feature space provides a meaningful basis for fake image detection, even though CLIP was never trained on this task.

6. Qualitative Results

6.1 Success Cases

The model reliably detects the following types of AI-generated content:

- **StyleGAN faces** ($P(\text{fake}) > 0.90$): The model consistently identifies GAN-generated face images with high confidence. StyleGAN faces exhibit characteristic smooth skin textures, unnatural symmetry, and subtle artifacts in hair and background boundaries that are captured in CLIP's feature space.
- **Stable Diffusion images** ($P(\text{fake}) > 0.85$): LDM-generated images are detected reliably. Diffusion models produce characteristic frequency-domain artifacts in fine detail regions that CLIP features implicitly encode.
- **Clear real photographs**: Natural photographs with rich texture, lighting variation, and typical camera noise are correctly classified as real with high confidence ($P(\text{fake}) < 0.15$).

6.2 Failure Cases and Analysis

The model exhibits systematic failure modes that provide insight into its decision boundary:

- **Modern smartphone photographs** (most common failure): Many real photos from modern smartphones are misclassified as fake. The root cause is distributional mismatch: the model's real training images are CIFAR-10 images (originally 32×32 , upscaled to 224×224 with Lanczos interpolation, creating characteristic blurriness) and specific face photograph styles. Modern smartphone photos are sharp, high-dynamic-range, and stylistically very different from these training real images.
- **HDR and heavily filtered photos**: Real photographs that have been processed with HDR enhancement, high-saturation Instagram-style filters, or artistic post-processing can score $P(\text{fake}) > 0.5$. The processing moves their CLIP embeddings toward the fake region.
- **Photorealistic AI images**: AI images specifically optimised for photorealism (e.g., Mid-journey v6) can occasionally score below the decision threshold, as their CLIP embeddings overlap with the real distribution.

Limitation and Future Fix: The real-image detection failures are fixable by adding more diverse real image training data. Adding COCO or ImageNet real images (which cover diverse modern photography styles) to the training set would significantly reduce the false positive rate on real photographs. This is left as future work due to time constraints.

7. Deployment

7.1 Web Application

The trained model is deployed as a live web application using Gradio 4.0 on Hugging Face Spaces (free tier):

Live Demo: <https://huggingface.co/spaces/hego2004/ai-image-detector>
Source Code: <https://github.com/hegoo2004/CSE429-AI-Image-Detection>

7.2 Deployment Architecture

A key deployment challenge was the 1.2 GB size of the full checkpoint. The solution was to separate the CLIP backbone (downloaded automatically from OpenAI on first run) from the linear head (stored in a 5 KB checkpoint file):

- **best.pt:** Contains only the 769 linear head parameters (5 KB)
- **CLIP backbone:** Downloaded from OpenAI's servers on first startup (≈ 1.7 GB, cached)
- **Inference pipeline:** Image upload $\rightarrow$ Resize to 224×224 $\rightarrow$ CLIP features $\rightarrow$ Linear head $\rightarrow$ P(fake) score $\rightarrow$ Verdict

7.3 User Interface

The web application provides:

- Image upload (drag-and-drop or file browser)
- Real/Fake verdict with confidence percentage
- P(fake) numerical score
- Confidence slider for visual representation
- Team information and methodology description

8. Implementation Details and Reproducibility

8.1 Software Stack

- Python 3.11.9 | PyTorch 2.5.1+cu121 | open-clip-torch 2.24.0
- scikit-learn 1.2+ | Pillow 12.0 | Gradio 4.0 | TensorBoard 2.13
- CUDA 12.1 | NVIDIA RTX 3050 6GB Laptop GPU

8.2 Unit Tests

21 unit tests in `tests/test_all.py` verify the implementation without requiring CLIP weights to be downloaded. A mock backbone is injected at test time. Key tests:

- Output shape is $[B, 1]$ for all batch sizes $\in \{1, 4, 32\}$
- Exactly 769 trainable parameters
- Gradients flow only to the head, never to the backbone
- Backbone stays in eval mode during training
- Output is L2-normalised (norm = 1.0)
- `NearestNeighbourDetector` raises `RuntimeError` before bank is built

Result: **21/21 tests pass** (runtime: 17.6s)

8.3 Reproduction Commands

```
git clone https://github.com/hegoo2004/CSE429-AI-Image-Detection
cd CSE429-AI-Image-Detection
pip install -r requirements.txt
python dataset.py --datasets cifake fakeface --max_per_class 5000
python train.py --epochs 20 --batch_size 32 --num_workers 2
python tests/test_all.py           # 21 tests should pass
python visualise_results.py        # generate all plots
python predict.py --image any_image.jpg
```

9. Conclusion and Future Work

9.1 Conclusion

This project successfully re-implements UniversalFakeDetect, demonstrating that a frozen CLIP-ViT-L/14 backbone combined with a 769-parameter linear classifier achieves **AUC = 0.9247** and **AP = 0.9283** on a combined test set covering latent diffusion (Stable Diffusion v1.4) and GAN (StyleGAN) generated images. Key findings:

1. **Parameter efficiency:** 769 trainable parameters achieve AUC = 0.92 while ~2M parameters in the Simple CNN achieve similar within-distribution accuracy but at the cost of generalisation.
2. **Stable training:** The 2–3% train-val accuracy gap across 20 epochs confirms that the minimal parameterisation effectively prevents overfitting.

3. **Strong fake detection:** 91.0% fake recall with $AP = 0.9283$ demonstrates reliable detection of AI-generated content at high precision.
4. **CLIP feature quality:** The clear real-vs-fake separation in the score distribution confirms that CLIP's feature space inherently separates real and AI-generated content, despite never being trained on this task.

9.2 Future Work

- **Dataset diversity:** Add COCO/ImageNet real images and DALL-E 3, SDXL, Midjourney v6 fake images for more robust training
- **Zero-shot evaluation:** Formally evaluate on generators not seen during training (DALL-E 3, SDXL, Midjourney v6) to verify cross-generator generalisation
- **Selective backbone fine-tuning:** Carefully tune the last 2–3 transformer blocks of ViT-L/14 while keeping earlier layers frozen
- **Adversarial robustness:** Evaluate against images specifically crafted to evade CLIP-based detection
- **Threshold calibration:** Apply Platt scaling or isotonic regression to calibrate the model's confidence scores for specific deployment contexts
- **Explainability:** Apply GradCAM or attention rollout to visualise which image regions drive the real-vs-fake decision

Author Contributions

- **Mohamed A. Elhageen (120220207) — Model Architecture & Implementation:** Designed and implemented the LinearProbingDetector and NearestNeighbourDetector in PyTorch, including the frozen CLIPBackbone wrapper, L2 normalisation, and all 21 unit tests.
- **Youssef A. Abo Wali (120220204) — Dataset Collection & Preprocessing:** Designed the download pipeline, implemented the preprocessing scripts (resize, normalise, stratified split), built the DataLoader with WeightedRandomSampler, and created the visualisation notebook.
- **Belal A. Abdelkarim (120220168) — Training Pipeline & Optimisation:** Implemented the full training loop with mixed-precision support, AdamW optimiser, cosine annealing scheduler, early stopping, gradient clipping, and TensorBoard logging.
- **Mohamed M. Kotb (120220042) — Evaluation & Results Analysis:** Implemented all evaluation metrics (AUC, F1, AP), confusion matrix analysis, ROC and PR curve generation, score distribution plots, and baseline comparisons.
- **Seif E. Ebeid (120220032) — Research Extension & Generalisation:** Designed the zero-shot evaluation framework for unseen generators (DALL-E 3, SDXL, Midjourney v6), implemented the failure analysis pipeline, and authored the extension section.

- **Omar I. Abdelhamid (120220132) — System Deployment & Web Application:** Built the Gradio web application, configured Hugging Face Spaces deployment with Git LFS, managed the GitHub repository, and implemented the command-line inference tool.

References

- [1] I. Goodfellow et al., “Generative Adversarial Networks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” *NeurIPS*, 2020.
- [3] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” *IEEE/CVF CVPR*, 2022.
- [4] U. Ojha, Y. Li, and Y. J. Lee, “Towards Universal Fake Image Detectors that Generalize Across Generative Models,” *IEEE/CVF CVPR*, 2023.
- [5] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, “CNN-Generated Images Are Surprisingly Easy to Spot... For Now,” *IEEE/CVF CVPR*, 2020.
- [6] Z. Liu, X. Qi, and P. H. S. Torr, “Global Texture Enhancement for Fake Face Detection in the Wild,” *IEEE/CVF CVPR*, 2020.
- [7] B. Liu, F. Yang, X. Bi, B. Xiao, W. Li, and S. Nefian, “Detecting Generated Images by Real Images,” *ECCV*, 2022.
- [8] J. J. Bird and A. Lotfi, “CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images,” *arXiv:2303.14126*, 2023.
- [9] seyoung62, “Real and Fake Face Detection,” Kaggle Dataset, 2019. Available: <https://www.kaggle.com/datasets/ciplab/real-and-fake-face-detection>
- [10] A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, 2021.
- [11] Ilharco, G. et al., “OpenCLIP,” GitHub: [mlfoundations/open_clip](https://github.com/mlfoundations/open_clip), 2021.